



Lyceum of the Philippines University – Cavite
College of Computer Studies

DCSN03C

Computer Programming 2

FINALS PROJECT

Text-Based RPG Game in Python

Project Documentation & Development Guide

Academic Year 2025–2026 | Finals Period

Table of Contents

1. Project Overview
2. Project Objectives & Learning Outcomes
3. Project Scope & Requirements
4. Technical Specifications
5. Object-Oriented Design Requirements
6. Game Mechanics Specification
7. Step-by-Step Development Guide
8. Project Documentation Template
9. Submission Guidelines
10. Grading Criteria Summary

Appendix A: Sample Class Diagram

Appendix B: Python Code Snippets

1. Project Overview

This document is the official project specification and development guide for the DCSN03C - Computer Programming 2 finals project. Students will design and develop a Text-Based RPG (Role-Playing Game) using Python. The project serves as a culminating assessment that integrates all major topics covered throughout the semester.

The text-based RPG must demonstrate competency in Object-Oriented Programming, input/output handling, variable management, conditional logic, and iterative control structures — all implemented in Python 3.

2. Project Objectives & Learning Outcomes

Upon completion of this project, students should be able to:

1. Apply Object-Oriented Design principles (encapsulation, inheritance, polymorphism) using Python classes.
2. Demonstrate proper use of Python variables, data types (int, float, str, bool, list, dict), and constants.
3. Implement conditional statements (if-elif-else) for decision-making game logic.
4. Utilize iterative statements (for, while) to create game loops and repetitive mechanics.
5. Handle user input via `input()` and produce formatted console output using `print()` and f-strings.
6. Produce standard IT project documentation following a structured format.
7. Practice software development with proper planning and testing in Python.

3. Project Scope & Requirements

3.1 General Requirements

- The game must run entirely in the console/terminal (text-based, no GUI).
- Minimum of three (3) distinct character classes or roles the player can choose from.
- A turn-based combat system against at least three (3) different enemy types.
- An inventory or item system (e.g., potions, weapons, armor) using Python lists or dictionaries.
- A simple progression system (e.g., leveling up, gaining stats).
- A win/lose condition with a clear endgame scenario.
- The game must be replayable (allow restart without relaunching the program).
- A group of 1 to 3 students

3.2 Functional Requirements

ID	Requirement	Description
FR-01	Character Creation	Player selects a class and names their character via input().
FR-02	Turn-Based Combat	Player and enemy take alternating turns: attack, defend, or use item.
FR-03	Inventory Management	Player can view, use, and acquire items stored in a list or dict.
FR-04	Enemy Encounters	Random (using the random module) or scripted encounters.
FR-05	Level Progression	Player gains experience and levels up with stat increases.
FR-06	Game Loop	A while loop drives exploration, encounters, and menus.
FR-07	Win/Lose Logic	Game ends with a boss fight or staged encounters with clear outcomes.

4. Technical Specifications

Specification	Details
Language	Python 3.8 or higher
Interface	Console / Terminal (text-based only; no tkinter, pygame, etc.)
Minimum Classes	At least five (5) user-defined classes
Design Patterns	Must use inheritance at least once; encapsulation via properties or naming conventions
Allowed Modules	random, time, os, abc (for abstract classes), json (for save/load). No third-party packages.
Input Handling	Must validate user input with try-except and handle invalid entries gracefully
File I/O (Bonus)	Save/load game state to a JSON file (optional, for bonus points)

5. Object-Oriented Design Requirements

This section outlines the specific OOP concepts that must be demonstrated in Python.

5.1 Required Classes (Minimum)

Class Name	Purpose	Key Attributes / Methods
Character	Base class (can be abstract via abc.ABC)	name, _hp, _attack_power, attack(), defend(), is_alive(), properties for hp
Player	Inherits Character; player-specific logic	level, exp, inventory: list, level_up(), use_item()
Enemy	Inherits Character; enemy-specific logic	enemy_type, drop_item(), get_exp_reward()
Item	Represents usable items	name, item_type, effect_value, use()
Game	Main game controller	player, enemies: list, game_loop(), start_battle(), display_menu()

5.2 OOP Concepts Checklist

Concept	How to Demonstrate in Python	Where
Encapsulation	Use _private or __mangled attributes with @property decorators for controlled access.	All classes
Inheritance	Player(Character) and Enemy(Character) using super().__init__().	Character hierarchy
Polymorphism	Override attack() or defend() in subclasses so they behave differently.	Player vs Enemy
Abstraction	Use abc.ABC and @abstractmethod to force subclasses to implement key methods.	Character base
Constructor	__init__() with parameters for each class; use super().__init__() in subclasses.	All classes
Magic Methods	__str__() or __repr__() for readable object output.	Character, Item

6. Game Mechanics Specification

6.1 Variables & Data Types

Students must demonstrate the following Python variable usage:

- int and float: HP, attack power, defense, experience, damage calculations.
- str: character names, item names, menu prompts, narrative text.
- bool: game state flags (is_alive, game_over, is_defending).
- list: inventory, enemy roster, available actions.
- dict: character stats mapping, item properties, enemy lookup tables.
- Constants (UPPER_SNAKE_CASE): MAX_HP, EXP_THRESHOLD, MIN_DAMAGE.

6.2 Conditional Statements

Conditional logic must appear in the following areas at minimum:

- Menu selection: if-elif-else chains for player action choices (Attack, Defend, Use Item, Run).
- Combat outcomes: damage calculation with conditionals for critical hits, misses, or type advantages.
- Win/Lose checks: if the player's HP drops to zero, trigger game over; if the boss is defeated, trigger victory.
- Input validation: check whether the user's input is valid before processing.

6.3 Iterative Statements

Loops must be present in:

- Game loop: a while loop that keeps the game running until the player wins, loses, or quits.
- Combat loop: a while loop that alternates turns between player and enemy until one side is defeated.
- Inventory traversal: a for loop with enumerate() to display and select items.
- Input retry: a while True loop with break on valid input to re-prompt on errors.

6.4 Input / Output

The program must:

- Accept keyboard input via input() for all player decisions.
- Display formatted output using f-strings for: HP bars or status displays, action menus, combat narration, and results.
- Handle edge cases using try-except blocks for ValueError on int(input()) and similar conversions.
- Optionally use os.system('cls' or 'clear') to clean the screen between game phases.

7. Step-by-Step Development Guide

Follow these phases to build your project systematically.

Phase 1: Planning & Design (Week 1)

1. Define your game theme and storyline (e.g., fantasy dungeon, space adventure, zombie apocalypse).
2. List all character classes you want (minimum 3 playable classes) with their stat differences.
3. Sketch a class diagram showing inheritance: Character → Player, Character → Enemy.
4. Write pseudocode or a flowchart for your main game loop.
5. Define your enemy types in a table: name, HP, attack, exp_reward.
6. List all items and their effects in a dictionary structure.

Phase 2: Core Classes (Week 2)

1. Create character.py with the Character base class using abc.ABC.
2. Create player.py with the Player subclass. Add level, exp, inventory, level_up(), use_item().
3. Create enemy.py with the Enemy subclass. Add enemy-specific attributes.
4. Create item.py with the Item class: name, item_type, effect_value, use().
5. Test each class in isolation using if `__name__ == '__main__'` blocks.

Phase 3: Game Logic (Week 3)

1. Create game.py with the Game class. Build the main menu: New Game, Instructions, Exit.
2. Implement character creation: input() for name, if-elif-else for class selection.
3. Implement the combat system with a while loop for turn-based battles.
4. Add inventory management: for loop with enumerate() to display and select items.
5. Implement experience gain and level_up() after defeating enemies.
6. Create at least 3 stages or encounters leading to a final boss.

Phase 4: Polish & Testing (Week 4)

1. Add input validation everywhere using try-except ValueError and while True loops.
2. Add flavor text, ASCII art, or formatted displays to enhance the experience.
3. Test all paths: winning, losing, each class, each item, each enemy.
4. Fix bugs and edge cases (e.g., using an item when inventory is empty).
5. Write your project documentation (see Section 8).
6. (Optional) Implement save/load using json.dump() and json.load() for bonus credit.

8. Project Documentation Template

Each group or individual must submit a project documentation following the structure below. This follows a standard IT documentation format.

Required Sections

#	Section	Content Description
1	Title Page	Project title, course code, section, group members, instructor name, date.
2	Table of Contents	Auto-generated or manually listed page references.
3	Introduction	Brief project background, purpose, and overview of the game.
4	Objectives	What the project aims to achieve (align with course learning outcomes).
5	Scope and Limitations	What the game includes and what it does not cover.
6	System Design	Class diagram (UML), flowcharts for game loop and combat, pseudocode.
7	Implementation Details	Description of each .py module, key methods, how OOP concepts are applied. Annotated code snippets.
8	Testing	Test cases table: input, expected output, actual output, pass/fail.
9	Screenshots / Sample Run	At least 5 screenshots showing key gameplay moments from the terminal.
10	Challenges & Lessons Learned	Difficulties encountered and how they were resolved.
11	Conclusion	Summary of accomplishments and potential improvements.
12	References	Books, websites, tutorials. Use APA or IEEE format.
13	Appendices	Complete Python source code listing, additional diagrams.

9. Submission Guidelines

Item	Details
Deadline	As announced by the instructor (strict; late submissions penalized).
Submission Format	Compressed folder (.zip): all .py files, documentation (.docx or .pdf), README.txt.
Naming Convention	DCSN03C_Finals_[LastName]_[FirstName].zip
File Structure	main.py (entry point), character.py, player.py, enemy.py, item.py, game.py.
Presentation	Each group/individual must demo the working game during the scheduled defense.
Plagiarism Policy	Copied code without attribution results in automatic failure.

10. Grading Criteria Summary

A detailed rubric is in the separate Grading Rubric document. Summary:

Criteria	Weight	Max Points
Object-Oriented Design	25%	25
Core Programming Constructs	25%	25
Game Functionality & Completeness	20%	20
Code Quality & Best Practices	10%	10
Project Documentation	10%	10
Presentation / Defense	10%	10
TOTAL	100%	100

Appendix A: Sample Class Diagram

Below is a simplified text representation. Students should produce a proper UML diagram using draw.io, Lucidchart, or hand-drawn.

```

+-----+
|  Character  (ABC)  |
+-----+
| - _name: str      |
| - _hp: int        |
| - _attack_power: int |
+-----+
| + attack(target) -> None | (abstract)
| + defend() -> None      |
| + is_alive() -> bool    |
| + hp (property)        |
+-----+
|
+-----+-----+
|               |
+-----+-----+
| -v            | -v            |
|  Player      |  Enemy        |
+-----+-----+
| - level: int   | - enemy_type: str |
| - exp: int     | - exp_reward: int |
| - inventory: list |
+-----+-----+
| + level_up()   | + drop_item()    |
| + use_item()   | + attack(target)  |
| + attack(target) |
+-----+-----+

+-----+-----+
|  Item        |  Game          |
+-----+-----+
| - name: str   | - player: Player |
| - item_type: str | - enemies: list   |
| - effect_value |
+-----+-----+
| + use(target)  | + game_loop()    |
| + __str__()    | + start_battle() |
|               | + display_menu() |
+-----+-----+

```

Appendix B: Python Code Snippets

The following snippets illustrate how key concepts are applied in Python.

B.1 Abstract Base Class with Encapsulation

```
from abc import ABC, abstractmethod

class Character(ABC):
    """Base class for all characters in the game."""

    def __init__(self, name: str, hp: int, attack_power: int):
        self._name = name
        self._hp = hp
        self._attack_power = attack_power

    @property
    def name(self) -> str:
        return self._name

    @property
    def hp(self) -> int:
        return self._hp

    @hp.setter
    def hp(self, value: int):
        self._hp = max(0, value)

    @abstractmethod
    def attack(self, target) -> None:
        pass

    def is_alive(self) -> bool:
        return self._hp > 0

    def __str__(self) -> str:
        return f"{self._name} (HP: {self._hp})"
```

B.2 Inheritance and Polymorphism

```
import random

class Player(Character):
    """Player character with leveling and inventory."""

    def __init__(self, name: str, hp: int, attack power: int):
        super().__init__(name, hp, attack_power)
        self.level = 1
        self.exp = 0
        self.inventory: list = []
```

```
def attack(self, target) -> None:
    damage = random.randint(self._attack_power - 2, self._attack_power + 2)
    damage = max(1, damage)
    print(f"{self.name} attacks {target.name} for {damage} damage!")
    target.hp -= damage

def level_up(self) -> None:
    self.level += 1
    self._hp += 10
    self._attack_power += 3
    print(f"{self.name} leveled up to level {self.level}!")
```

B.3 Game Loop with Iteration and Conditionals

```
game_over = False

while not game_over:
    display_menu()
    choice = get_valid_input(1, 4)

    if choice == 1:
        start_battle(player, current_enemy)
    elif choice == 2:
        show_inventory(player)
    elif choice == 3:
        show_stats(player)
    elif choice == 4:
        print("Thanks for playing!")
        game_over = True
```

B.4 Input Validation with try-except and while Loop

```
def get_valid_input(min_val: int, max_val: int) -> int:
    """Keep prompting until the user enters a valid integer in range."""
    while True:
        try:
            choice = int(input(f"Enter choice ({min_val}-{max_val}): "))
            if min_val <= choice <= max_val:
                return choice
            else:
                print(f"Please enter a number between {min_val} and {max_val}.")
        except ValueError:
            print("Invalid input. Please enter a number.")
```

B.5 Inventory Display with for and enumerate()

```
def show_inventory(player: Player) -> None:
    if not player.inventory:
        print("Your inventory is empty.")
```

```
    return

    print("\n== INVENTORY ==")
    for i, item in enumerate(player.inventory, start=1):
        print(f"    {i}. {item}")
    print("=====")
```

B.6 Suggested File Structure

```
DCSN03C_Finals_YourName/
├── main.py                # Entry point: from game import Game; Game().run()
├── character.py           # Character (ABC)
├── player.py              # Player(Character)
├── enemy.py               # Enemy(Character)
├── item.py                # Item class
├── game.py                # Game controller
├── README.txt             # How to run: python main.py
├── docs/
│   └── Documentation.docx
```

End of Document